

Algos condensed and explained

salimek

June 17, 2026

1 Pattern

TITLE
Explanation of the algorithm.
Pseudo-code Algorithm(...) instruction 1 instruction 2 return result
Time Complexity: $O(n)$
Space Complexity: $O(1)$
Recurrence Formula: $T(n) = 2T\left(\frac{n}{2}\right) + n$

2 Algos (everything may not be finished and explained)

INSERTION SORT

Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time. At each iteration, the current element is inserted into its correct position inside the already sorted part of the array.

Pseudo-code

Insertion-Sort(A, n)

```
for  $j = 2$  to  $n$ 
     $key = A[j]$ 
     $i = j - 1$ 
```

```
    while  $i > 0$  and  $A[i] > key$ 
         $A[i + 1] = A[i]$ 
         $i = i - 1$ 
```

```
     $A[i + 1] = key$ 
```

Time Complexity: $\Theta(n^2)$ worst/average, $\Theta(n)$ best case

Space Complexity: $O(1)$

Recurrence Formula: N/A

Merge Sort

Divide the array into two halves, recursively sort each half, and merge the sorted halves. The objective is to sort an array efficiently using the divide-and-conquer strategy.

Pseudo-code

```
MERGE-SORT(A, p, r)

    if p < r
        q = rounddown((p+r)/2)

        MERGE-SORT(A, p, q)
        MERGE-SORT(A, q+1, r)

        MERGE(A, p, q, r)

MERGE(A, p, q, r)

    n1 = q-p+1
    n2 = r-q

    create arrays L[1...n1+1], R[1...n2+1]

    for i = 1 to n1
        L[i] = A[p+i-1]

    for j = 1 to n2
        R[j] = A[q+j]

    L[n1+1] = 8
    R[n2+1] = 8

    i = 1
    j = 1

    for k = p to r
        if L[i] <= R[j]
            A[k] = L[i]
            i = i + 1
        else
            A[k] = R[j]
            j = j + 1
```

Time Complexity: $\Theta(n \log n)$ in all cases.

Space Complexity: $O(n)$ auxiliary space.

Recurrence Formula: $T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$ by Master Theorem (case 2).

Maximum Subarray Optimized

Find the continuous subarray with the maximum possible sum. The algorithm uses divide and conquer by splitting the array, solving both halves recursively, and checking the subarray crossing the middle.

Pseudo-code

```
FIND-MAXIMUM-SUBARRAY(A, low, high)

    if high == low
        return (low, high, A[low])

    mid = (low + high) / 2

    left = FIND-MAXIMUM-SUBARRAY(A, low, mid)
    right = FIND-MAXIMUM-SUBARRAY(A, mid+1, high)

    cross = FIND-MAX-CROSSING(A, low, mid, high)

    if left-sum >= right-sum and left-sum >= cross-sum
        return (left-low, left-high, left-sum)

    elseif right-sum >= left-sum and right-sum >= cross-sum
        return (right-low, right-high, right-sum)

    else
        return (cross-low, cross-high, cross-sum)

FIND-MAX-CROSSING(A, low, mid, high)

    left-sum = -8
    sum = 0

    for i = mid downto low
        sum = sum + A[i]

        if sum > left-sum
            left-sum = sum
            max-left = i

    right-sum = -8
    sum = 0

    for j = mid+1 to high
        sum = sum + A[j]

        if sum > right-sum
            right-sum = sum
            max-right = j

    return (max-left, max-right, left-sum + right-sum)
```

Time Complexity: $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ by Master Theorem.

Space Complexity: $O(\log n)$ auxiliary space due to recursion stack.

Recurrence Formula: $T(n) = 2T(n/2) + \Theta(n)$

Maximum Subarray Brute Force

Checks every possible continuous subarray and keeps the one with the largest sum. The algorithm uses an exhaustive search approach.

Pseudo-code

```
MAXIMUM-SUBARRAY-SLOW(A[1..n])  
  
    B.val = 8  
    B.i = 1  
    B.j = n  
  
    for i = 1 to n  
        tmp = 0  
        for j = i to n  
            tmp = tmp + A[j]  
            if tmp > B.val  
                B.val = tmp  
                B.i = i  
                B.j = j  
  
    return (B.i, B.j, B.val)
```

Time Complexity: $\Theta(n^2)$

Space Complexity: $\Theta(n)$ input space + $\Theta(1)$ auxiliary space.

Recurrence Formula: No recurrence formula (iterative brute force algorithm).

Matrix Multiplication Naive

Multiply two square matrices using the standard triple nested loop approach. Each element of the result matrix is computed by the dot product of a row and a column.

Pseudo-code

```
SQUARE-MAT-MULT(A, B, n)
    create matrix C of size n x n
    for i = 1 to n
        for j = 1 to n
            C[i][j] = 0
            for k = 1 to n
                C[i][j] = C[i][j] + A[i][k] * B[k][j]
    return C
```

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$ for the result matrix.

Recurrence Formula: $T(n) = 8T(n/2) + \Theta(n^2) \Rightarrow T(n) = \Theta(n^3)$
by Master Theorem (case 1).

Recursive Matrix Multiplication

Multiply two square matrices using a divide-and-conquer approach. The matrices are divided into smaller submatrices and recursively multiplied.

Pseudo-code

```
REC-MAT-MULT(A, B, n)

    create matrix C of size n x n

    if n == 1
        C11 = A11 * B11

    else
        partition A, B, C into n/2 x n/2 blocks

        C11 = REC-MAT-MULT(A11, B11, n/2)
              + REC-MAT-MULT(A12, B21, n/2)

        C12 = REC-MAT-MULT(A11, B12, n/2)
              + REC-MAT-MULT(A12, B22, n/2)

        C21 = REC-MAT-MULT(A21, B11, n/2)
              + REC-MAT-MULT(A22, B21, n/2)

        C22 = REC-MAT-MULT(A21, B12, n/2)
              + REC-MAT-MULT(A22, B22, n/2)

    return C
```

Time Complexity: $T(n) = 8T(n/2) + \Theta(n^2) \Rightarrow O(n^3)$ by Master Theorem.

Space Complexity: $O(n^2)$ for storing matrices.

Recurrence Formula: $T(n) = 8T(n/2) + \Theta(n^2)$

Strassen's Algorithm

Multiply two matrices using divide and conquer while reducing the number of recursive multiplications from 8 to 7. The algorithm uses linear combinations of submatrices to improve the complexity.

Pseudo-code

```
STRASSEN-MAT-MULT(A, B, n)
    create matrix C of size n x n
    if n == 1
        C11 = A11 * B11
    else
        partition A, B, C into n/2 x n/2 blocks

        M1 = (A11 + A22) * (B11 + B22)
        M2 = (A21 + A22) * B11
        M3 = A11 * (B12 - B22)
        M4 = A22 * (B21 - B11)
        M5 = (A11 + A12) * B22
        M6 = (A21 - A11) * (B11 + B12)
        M7 = (A12 - A22) * (B21 + B22)

        C11 = M1 + M4 - M5 + M7
        C12 = M3 + M5
        C21 = M2 + M4
        C22 = M1 - M2 + M3 + M6

    return C
```

Time Complexity: $T(n) = 7T(n/2) + \Theta(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

Space Complexity: $O(n^2)$ for storing matrices.

Recurrence Formula: $T(n) = 7T(n/2) + \Theta(n^2)$

Karatsuba Multiplication

Multiply two large integers faster than the classical multiplication method. The algorithm reduces the number of recursive multiplications from 4 to 3 using a divide-and-conquer strategy.

Pseudo-code

```
KARATSUBA-MULT(x, y, n)

    if n == 1
        return x * y

    else
        split x into xH, xL
        split y into yH, yL

        p1 = KARATSUBA-MULT(xH, yH, n/2)
        p2 = KARATSUBA-MULT(xL, yL, n/2)
        p3 = KARATSUBA-MULT(
            xH + xL,
            yH + yL,
            n/2)

        return p1*b^n
            + (p3 - p1 - p2)*b^(n/2)
            + p2
```

Time Complexity: $T(n) = 3T(n/2) + \Theta(n) \Rightarrow \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

Space Complexity: $O(1)$ auxiliary space.

Recurrence Formula: $T(n) = 3T(n/2) + \Theta(n)$

MAX-HEAPIFY

Restore the max-heap property at a given node by comparing it with its children. If a child is larger, swap values and continue recursively down the heap.

Pseudo-code

```
MAX-HEAPIFY(A, i, n)
    l = LEFT(i)
    r = RIGHT(i)

    if l <= n and A[l] > A[i]
        largest = l
    else
        largest = i

    if r <= n and A[r] > A[largest]
        largest = r

    if largest != i
        exchange A[i] with A[largest]
        MAX-HEAPIFY(A, largest, n)
```

Time Complexity: $\Theta(\text{height}(i)) = O(\log n)$

Space Complexity: $O(\log n)$ due to recursion stack.

Recurrence Formula: $T(n) = T(n/2) + \Theta(1) \Rightarrow T(n) = O(\log n)$

Build Heap

Transform an unordered array into a valid max-heap. The algorithm applies MAX-HEAPIFY bottom-up on all internal nodes to restore the heap property efficiently.

Pseudo-code

```
BUILD-MAX-HEAP(A, n)
  for i = n/2 downto 1
    MAX-HEAPIFY(A, i, n)
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$ auxiliary space.

Recurrence Formula: No recurrence formula (bottom-up iterative algorithm).

Heapsort

Sort an array using a max-heap structure. The algorithm first builds a max-heap, then repeatedly extracts the maximum element and restores the heap property.

Pseudo-code

```
HEAPSORT(A, n)
    BUILD-MAX-HEAP(A, n)

    for i = n downto 2
        exchange A[1] with A[i]
        MAX-HEAPIFY(A, 1, i-1)
```

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$ auxiliary space.

Recurrence Formula: No recurrence formula (iterative heap-based algorithm).

Heap Extract Max

Remove and return the maximum element of a max-heap. The root is replaced by the last element, then MAX-HEAPIFY is used to restore the heap property.

Pseudo-code

```
HEAP-EXTRACT-MAX(A, n)
    if n < 1
        error "heap underflow"

    max = A[1]
    A[1] = A[n]
    n = n - 1

    MAX-HEAPIFY(A, 1, n)

    return max
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$ auxiliary space.

Recurrence Formula: $T(n) = T(n/2) + \Theta(1) \Rightarrow O(\log n)$